Kerim Turak

Ankara Yıldırım Beyazıt University
EEE Department- Spring, 20-21
EE226 Introduction to Microprocessors Laboratory

Universal Asynchronous Receiver-Transmitter (UART)

Explanation and Implementation Video: https://youtu.be/iuwGk6QFmYg
NOTE : In my code some external(from Valvano Labs) header file are added therefore to run my code download my code from following link https://drive.google.com/file/d/10x43SHQybsE6DWa_o6kq_GwCKn0bgKtg/view?usp=sharing

1. Introduction

In this lab, we used the ultrasonic distance sensor (HC-SR04) and USB to TLL device to understand the UART communication with working principle of HC-SR04 and USB to TLL device communication logic (by using RX, TX connection)

HC-SR04

At its core, the HC-SR04 Ultrasonic distance sensor consists of two ultrasonic transducers. The one acts as a transmitter which converts electrical signal into 40 KHz ultrasonic sound pulses. The receiver listens for the transmitted pulses. If it receives them, it produces an output pulse whose width can be used to determine the distance the pulse travelled. According to HC-SR04 working principle I used following formula to calculate the distance.

Distance = Speed x Time

USB TO TLL

The **PL-2303 chip** provides a convenient solution for connecting a UART-like full-duplex asynchronous serial device to any Universal Serial Bus (USB) capable host. The PL-2303 highly compatible drivers could successfully simulate generic COM ports on most operating systems.

Setting up the Algorithm and Writing the Code

Firstly, we need to know which module or part of the launchpad will be used in this task. We will mainly use UART, TIMER and we have to initialize necessary setting for this task. I create a template as following.

```c
/*header files for TM4C123 */
#include <stdint.h>
#include "UART.h"
#include "tm4c123gh6pm.h"
#include "TM4C123GH6PM.h"
#include <stdio.h>

/*Function prototype for Timer0A and UART module initialization */
uint32_t Measure_distance(void);
void Timer0ACapture_init(void);
void Delay_MicroSecond(int time);
void UART5_init(void);
void UART5_Transmitter(unsigned char data);
void printstring(char *str);
void Delay(unsigned long counter);
void PortFInit(void);
```

I used some library and header file to complete the lab task. After my code is done I got some error and I found the solution by adding the "UART.h". In addition, I use Valvano's header file "tm4c123gh6pm.h" to use mnemonic label.

- uint32_t Measure_distance(void); for distance measurement
- void Timer0ACapture_init(void); timer
- void Delay_MicroSecond(int time); delay
- void UART5_init(void);
- void UART5_Transmitter(unsigned char data);UART initialize
- void printstring(char *str); for printing string on putty
- void Delay(unsigned long counter);
- void PortFInit(void); portf initialize

There are a number of variable that I use ;

```c
/* global variables to store and display distance in cm */
uint32_t time; /*stores pulse on time */
uint32_t distance; /* stores measured distance value */
char mesg[20];  /* string format of distance value */
```

"mesg" here holding a string as array in memory and it will print using pointer logic on putty screen.

```c
int main(void)
{
PortFInit();
Timer0ACapture_init();  /*initialize Timer0A in edge edge time */
UART5_init(); /* initialize UART5 module to transmit data to computer */
  while(1)
  {
time = Measure_distance(); /* take pulse duration measurement */
distance = (time * 10625)/10000000; /* convert pulse duration into distance */
sprintf(mesg, "\r\nDistance = %d cm", distance); /*convert float type distance data into string */
printstring(mesg); /*transmit data to computer */
Delay(2000);
if(distance >= 30 )
  GPIO_PORTF_DATA_R = 0x00;
else if (distance < 30 && distance > 20)
  GPIO_PORTF_DATA_R = 0x08;
else if (distance < 20 && distance > 10)
  GPIO_PORTF_DATA_R = 0x04;
else
  GPIO_PORTF_DATA_R = 0x02;
}
}
```

In the main code block, I did necessary calculation and led order logic as above.
Last important part of my code is printing part by using pointer logic as following.

```
void printstring(char *str)
{
  while(*str)
  {
    UART5_Transmitter(*(str++));
  }
}
```

Other parts are standard format like enabling ports and assigning necessary number to registers.

2. Result and Discussion

To complete this lab, I first learned the USB TO PLL logic. I learned how to make two important connections such as RX, TX and communication between 2 UART units. With the ultrasonic distance sensor, I better understood how the distance is calculated and how the wave frequency is used when calculating this distance. I have understood the relationship between the sensor's receiver and transmitter, and the point to note here is that the sensor does not work absolutely well with 3.3V. I found out that the ideal operating voltage of the sensor is 5V.

3. Conclusion

I understood better how a wave from the TX of the distance sensor hits an obstacle and how to find the distance as wave speed times the time by taking the RX back, and how to set and use the UART communication and, in addition, how the clock is set.

4. References

a. https://deepbluembedded.com/usb-ttl-converter-tutorial-pc-control-for-microcontrollers/
b. https://www.electronicwings.com/sensors-modules/ultrasonic-module-hc-sr04
c. https://www.microcontrollertips.com/principle-applications-limitations-ultrasonic-sensors-faq/
d. https://herenkeskin.com/uart-nedir-ve-nasil-calisir/
e. https://en.wikipedia.org/wiki/Universal_asynchronous_receiver-transmitter
f. https://stackoverflow.com/questions/14306867/print-out-elements-of-an-array-of-strings-in-c
g. https://www.includehelp.com/c-programs/create-and-print-array-of-strings.aspx

Appendix

```
/*header files for TM4C123 */
#include <stdint.h>
#include "UART.h"
#include "tm4c123gh6pm.h"
#include "TM4C123GH6PM.h"
#include <stdio.h>

/*Function prototype for Timer0A and UART module initialization */
uint32_t Measure_distance(void);
void Timer0ACapture_init(void);
void Delay_MicroSecond(int time);
void UART5_init(void);
void UART5_Transmitter(unsigned char data);
void printstring(char *str);
void Delay(unsigned long counter);
void PortFInit(void);

/* global variables to store and display distance in cm */
uint32_t time; /*stores pulse on time */
uint32_t distance; /* stores measured distance value */
char mesg[20];  /* string format of distance value */

/* main code to take distance measurement and send data to UART terminal */
int main(void)
{
PortFInit();
Timer0ACapture_init();  /*initialize Timer0A in edge time */
UART5_init(); /* initialize UART5 module to transmit data to computer */
        while(1)
        {
time = Measure_distance(); /* take pulse duration measurement */
distance = (time * 10625)/10000000; /* convert pulse duration into distance */
sprintf(mesg, "\r\nDistance = %d cm", distance); /*convert float type distance data into string
*/
printstring(mesg); /*transmit data to computer */
Delay(2000);
if(distance >= 30 )
        GPIO_PORTF_DATA_R = 0x00;
else if (distance < 30 && distance > 20)
        GPIO_PORTF_DATA_R = 0x08;
else if (distance < 20 && distance > 10)
        GPIO_PORTF_DATA_R = 0x04;
else
        GPIO_PORTF_DATA_R = 0x02;
}
}

/* This function captures consecutive rising and falling edges of a periodic signal */
```

```
/* from Timer Block 0 Timer A and returns the time difference (the period of the signal). */
uint32_t Measure_distance(void)
{
   int lastEdge, thisEdge;

          /* Given 10us trigger pulse */
          GPIO_PORTA_DATA_R &= ~(1<<4); /* make trigger  pin high */
          Delay_MicroSecond(10); /*10 seconds delay */
          GPIO_PORTA_DATA_R |= (1<<4); /* make trigger  pin high */
          Delay_MicroSecond(10); /*10 seconds delay */
          GPIO_PORTA_DATA_R &= ~(1<<4); /* make trigger  pin low */

         while(1)
         {
   TIMER0_ICR_R = 4;           /* clear timer0A capture flag */
   while((TIMER0_RIS_R & 4) == 0) ;    /* wait till captured */
          if(GPIO_PORTB_DATA_R & (1<<6)) /*check if rising edge occurs */
                 {
   lastEdge = TIMER0_TAR_R;     /* save the timestamp */
                 /* detect falling edge */
   TIMER0_ICR_R = 4;           /* clear timer0A capture flag */
   while((TIMER0_RIS_R & 4) == 0) ;    /* wait till captured */
   thisEdge = TIMER0_TAR_R;     /* save the timestamp */
                 return (thisEdge - lastEdge); /* return the time difference */
                 }
         }
}

/* Timer0A initialization function */
/* Initialize Timer0A in input-edge time mode with up-count mode */
void Timer0ACapture_init(void)
{
   SYSCTL_RCGCTIMER_R |= 1;     /* enable clock to Timer Block 0 */
   SYSCTL_RCGCGPIO_R |= 2;      /* enable clock to PORTB */

   GPIO_PORTB_DIR_R &= ~0x40;        /* make PB6 an input pin */
   GPIO_PORTB_DEN_R |= 0x40;         /* make PB6 as digital pin */
   GPIO_PORTB_AFSEL_R |= 0x40;       /* use PB6 alternate function */
   GPIO_PORTB_PCTL_R &= ~0x0F000000;  /* configure PB6 for T0CCP0 */
   GPIO_PORTB_PCTL_R |= 0x07000000;

          /* PB2 as a digital output signal to provide trigger signal */
          SYSCTL_RCGCGPIO_R |= 1;      /* enable clock to PORTA */
          GPIO_PORTA_DIR_R |=(1<<4);         /* set PB4 as a digial output pin */
          GPIO_PORTA_DEN_R |=(1<<4);         /* make PB4 as digital pin */

   TIMER0_CTL_R &= ~1;          /* disable timer0A during setup */
   TIMER0_CFG_R = 4;            /* 16-bit timer mode */
   TIMER0_TAMR_R = 0x17;        /* up-count, edge-time, capture mode */
   TIMER0_CTL_R |=0x0C;         /* capture the rising edge */
```

```
   TIMER0_CTL_R |= (1<<0);          /* enable timer0A */
}



/* Create one microsecond second delay using Timer block 1 and sub timer A */

void Delay_MicroSecond(int time)
{
   int i;
   SYSCTL_RCGCTIMER_R |= 2;     /* enable clock to Timer Block 1 */
   TIMER1_CTL_R = 0;            /* disable Timer before initialization */
   TIMER1_CFG_R = 0x04;         /* 16-bit option */
   TIMER1_TAMR_R = 0x02;        /* periodic mode and down-counter */
   TIMER1_TAILR_R = 16 - 1;  /* TimerA interval load value reg */
   TIMER1_ICR_R = 0x1;          /* clear the TimerA timeout flag */
   TIMER1_CTL_R |= 0x01;        /* enable Timer A after initialization */

   for(i = 0; i < time; i++)
   {
      while ((TIMER1_RIS_R & 0x1) == 0) ;      /* wait for TimerA timeout flag */
      TIMER1_ICR_R = 0x1;      /* clear the TimerA timeout flag */
   }
}
void UART5_init(void)
{
           SYSCTL_RCGCUART_R |= 0x20;  /* enable clock to UART5 */
   SYSCTL_RCGCGPIO_R |= 0x10;  /* enable clock to PORTE for PE4/Rx and RE5/Tx */
   /* UART0 initialization */
   UART5_CTL_R = 0;         /* UART5 module disbable */
   UART5_IBRD_R= 104;       /* for 9600 baud rate, integer = 104 */
   UART5_FBRD_R = 11;       /* for 9600 baud rate, fractional = 11*/
   UART5_CC_R = 0;          /*select system clock*/
   UART5_LCRH_R = 0x60;     /* data lenght 8-bit, not parity bit, no FIFO */
   UART5_CTL_R = 0x301;     /* Enable UART5 module, Rx and Tx */

   /* UART5 TX5 and RX5 use PE4 and PE5. Configure them digital and enable alternate
function */
   GPIO_PORTE_DEN_R = 0x30;      /* set PE4 and PE5 as digital */
   GPIO_PORTE_AFSEL_R = 0x30;    /* Use PE4,PE5 alternate function */
   GPIO_PORTE_AMSEL_R = 0;    /* Turn off analg function*/
   GPIO_PORTE_PCTL_R = 0x00110000;     /* configure PE4 and PE5 for UART */
}
void UART5_Transmitter(unsigned char data)
{
   while((UART5_FR_R & (1<<5)) != 0); /* wait until Tx buffer not full */
   UART5_DR_R = data;                 /* before giving it another byte */
}

void printstring(char *str)
```

```
{
  while(*str)
        {
                UART5_Transmitter(*(str++));
        }
}
void Delay(unsigned long counter)
{
        unsigned long i = 0;

        for(i=0; i< counter*1000; i++);
}
/* This function is called by the startup assembly code to perform system specific
initialization tasks. */
/* This function is called by the startup assembly code to perform system specific
initialization tasks. */
void PortFInit(void) {
   SYSCTL_RCGC2_R    |= 0x00000020;     // 1) F clock
   GPIO_PORTF_LOCK_R  = 0x4C4F434B;     // 2) unlock Port F
   GPIO_PORTF_CR_R   |= 0x1F;           // allow changes to PF4-PF0
   GPIO_PORTF_AMSEL_R = 0x00;           // 3) disable analog function
   GPIO_PORTF_PCTL_R  = 0x00;           // 4) GPIO clear bit PCTL
   GPIO_PORTF_DIR_R  |= 0x0E;           // 5) PF4, PF0 inputs. PF3, PF2, PF1 outputs
   GPIO_PORTF_AFSEL_R = 0x00;           // 6) no alternate function
   GPIO_PORTF_PUR_R  |= 0x11;           // enable pullup resistors on PF4, PF0
   GPIO_PORTF_DEN_R  |= 0x1F;           // 7) enable digital pins PF4-PF0
}
```